WEEK 08

JSON values and dictionary checks

This week: learn, practise, build

01 · Python

JSON values and dictionary checks
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Reports and donor review queue
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

JSON values and dictionary checks
Concept → trace a small example → explain its behavior

JSON text, Python values and validation order

Parsing

JSON text becomes a Python value with a specific type.

Shape before fields

Check for a dictionary before reading count.

Contract before conversion

An exact positive integer contract excludes True, "4" and 4.0.

Valid JSON syntax does not guarantee a valid input value.

Decode JSON, then inspect the Python value

JSON is text

json.loads converts JSON text into Python values: object → dict, array → list, null → None.

Check the shape first

Valid JSON is not always a dictionary. Check the top-level type before using .get().

Check each required field

Missing, wrong-type and out-of-range values need explicit handling. JSON true becomes Python True.

Understand the behavior before changing the example.

JSON text becomes Python values

1

Decode

json.loads(text)
JSON text → Python value

2

Inspect shape

Is the result a dictionary, a list or None?

3

Read and check

Use .get() on a dictionary. Check type and range.

Parsing, inspecting shape and validating fields are separate steps.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
json.loads(text)	Parse JSON text	Obtain Python values
isinstance(x, dict)	Check the shape	Before dictionary methods
mapping.get(key)	Read a field	Handle a missing key
type(x) is int	Check an exact integer	Exclude bool, then check range

Python Basics · sections 05, 09, 12, 13 · use only when you need a lookup.

Decode one JSON value at a time

```
01 import json
02 print(type(json.loads('{"count": 4}')).__name__)
03 print(type(json.loads('[]')).__name__)
04 print(type(json.loads('null')).__name__)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Check shape before reading a key

```
01 import json
02 candidate = json.loads('[]')
03 if type(candidate) is not dict:
04     print("Object required")
05 else:
06     print(candidate.get("count"))
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Distinguish a missing key from an invalid value

```
01 import json
02 candidate = json.loads('{"count": null}')
03 print("count" in candidate)
04 value = candidate.get("count")
05 print(value)
06 print(type(value) is int and value > 0)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Catch malformed JSON specifically

```
01 import json
02 text = "{"
03 try:
04     value = json.loads(text)
05 except json.JSONDecodeError:
06     print("Invalid JSON")
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

JSON parsing does not guarantee a dictionary

```
01 import json
02 for text in ['{"count": 4}', '[4]', 'null']:
03     value = json.loads(text)
04     print(repr(value), type(value).__name__)
```

1. Expected types: dict, list, NoneType.
2. All three inputs are valid JSON.
3. Only the dictionary supports field lookup with .get().

Predict → execute → explain the value at each step.

JSON values and exact Python types

```
01 import json
02 candidate = json.loads('{"containers": true}')
03 count = candidate.get("containers")
04 valid = type(count) is int and count > 0
05 print(count, valid)
06 assert valid is False
```

1. json.loads() converts JSON values.
2. type(count) is int excludes bool.
3. Check the range after the type.

JSON true becomes bool, which is different from an exact int.

Dictionary lookup and a rejected transition

```
def next_state(state, event):
    allowed = {"reserved": "collected",
               "collected": "delivered"}
    if allowed.get(state) != event:
        raise ValueError("out of order or repeated")
    return event
state = "reserved"
try:
    state = next_state(state, "delivered")
except ValueError:
    pass  # Deliberate failed transition.
assert state == "reserved"
```

1. Use .get() to look up the allowed value.
2. Raise when the condition fails.
3. Check the value after except.

A raised exception prevents assignment of a new result.

Hands-on practice

Week_08.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Parse JSON, inspect Python types and check the shape before reading fields.

Project studio

Reports and donor review queue
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

The coordinator needs to distinguish promises from actual events.

2

This week, you will

Compare pickup records and review incoming messages before using their claims.

3

A useful result

The coordinator knows which pickups need follow-up.

Use the supplied starting files. Focus on one useful result and check it.

Use events to explain each lifecycle stage

1

Reserved

Accepted request
Availability reduced

2

Collected

Explicit collection event
For an accepted request

3

Delivered

Explicit delivery event
Collection must come first

These measures overlap. Do not add them as separate amounts.

Practice offline, then record required HTTP evidence

Offline simulation

No socket is opened. Tests exercise candidate validation and a deliberately raised failure.

Required classroom HTTP

Set the flag True. Send ID and original text; record actual responses and the six-message queue.

Simulation tests handling; only an actual request tests the HTTP path.

GROUP BY: aggregate related rows

```sql
SELECT batch_id, SUM(containers) AS containers,
       SUM(containers * ppc) AS planned_portions
FROM lines
GROUP BY batch_id
ORDER BY batch_id;
```

1. Miniature T10: three containers, nine portions
2. Miniature T20: one container, four portions
3. Group by request_id to reconcile another way

The grouping key determines what one output row means.

EXISTS: filter without multiplying rows

```sql
SELECT COALESCE(SUM(r.planned_portions), 0)
FROM reservations r
WHERE EXISTS (
  SELECT 1 FROM events e
  WHERE e.request_id = r.request_id
    AND e.event_type = 'collected'
);
```

1. The inner query tests for a match.
2. Each outer row contributes once.
3. COALESCE replaces a null total with zero.

EXISTS filters outer rows without multiplying them.

try / except: retain failure information

```
from urllib.error import HTTPError
queue = []
try:
    candidate = fetch(message_id)  # supplied context
except HTTPError as error:
    queue.append({"message_id": message_id,
                  "reason": f"HTTP {error.code}",
                  "next_action": "coordinator follow-up"})
```

1. fetch is supplied by the example context.
2. except binds the HTTPError to error.
3. append() adds a structured failure record.

Catch a specific failure and retain information needed by the caller.

Your implementation TODO

Supplied starting point

Canonical CSV/database, six messages, offline fetch and local HTTP replay service are supplied.

You implement / do

Implement grouped reports and field/shape checks. Retain source, outcome, reasons and action for M1–M6; run offline and actual local HTTP evidence.

Check the evidence

Metrics: offered 94, available 34, reserved 60, collected 42, delivered 29. Reconcile 34+18+13+29=94; M6 HTTP503 needs follow-up; intake leaves DB unchanged.

Keep offline and actual HTTP evidence separate; pitch a small Week 9 change.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Implement metrics and queue; record actual local HTTP.
3 · Check	94/34/60/42/29; M6 follow-up; intake leaves DB unchanged.
4 · Explain	Show your code, its actual result and why it behaves that way.

Keep offline and actual HTTP evidence separate; pitch a small Week 9 change.

Further reading in English

1. ## Python urllib.request documentation

   https://docs.python.org/3/library/urllib.request.html

2. ## Python json documentation

   https://docs.python.org/3/library/json.html

3. ## SQLite aggregate functions

   https://www.sqlite.org/lang_aggfunc.html

Read one relevant section, then bring an example and a question.